



Esplorazione di grafi non orientati

Martedì 5 maggio 2026

Algoritmi su grafi

- + Una tecnica per implementarli prevede di disaccoppiare l'implementazione dalla rappresentazione.

- + Esempio: ricerca

```
public class Search
```

```
Search(Graph G, int s) find vertices connected to a source vertex s
```

```
boolean marked(int v) is v connected to s?
```

```
int count() how many vertices are connected to s?
```

```
public class TestSearch
{
    public static void main(String[] args)
    {
        Graph G = new Graph(new In(args[0]));
        int s = Integer.parseInt(args[1]);
        Search search = new Search(G, s);

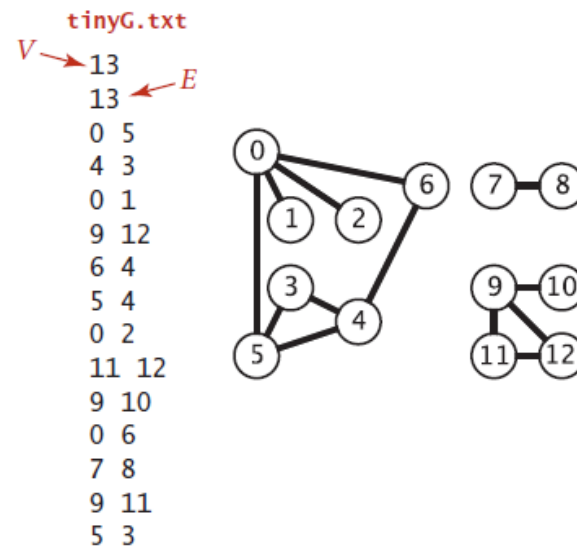
        for (int v = 0; v < G.V(); v++)
            if (search.marked(v))
                StdOut.print(v + " ");
        StdOut.println();

        if (search.count() != G.V())
            StdOut.print("NOT ");
        StdOut.println("connected");
    }
}
```

Esempio

```
% java TestSearch tinyG.txt 0
0 1 2 3 4 5 6
NOT connected

% java TestSearch tinyG.txt 9
9 10 11 12
NOT connected
```



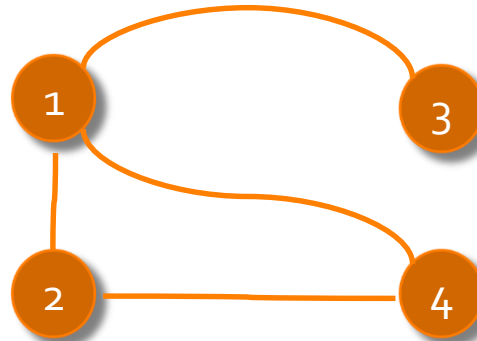
Esercizi

- + Progettare un algoritmo, implementato attraverso un opportuno metodo, che dato un grafo non orientato $G = (V, E)$, ne calcoli il grafo che ne rappresenta il suo “quadrato”. Il quadrato del grafo è il grafo $G^2 = (V, E^2)$ tale che $\{u, w\} \in E^2$ se e solo se $\exists v : \{u, v\} \in E \wedge \{v, w\} \in E$. In altre parole, se esiste un percorso di due archi fra i nodi u e w .
- + Affrontare lo stesso problema utilizzando l'implementazione di un grafo mediante matrice di adiacenza

Matrici di adiacenza e numero di cammini

+ Dato il grafo rappresentato dalla seguente matrice:

$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix}$$



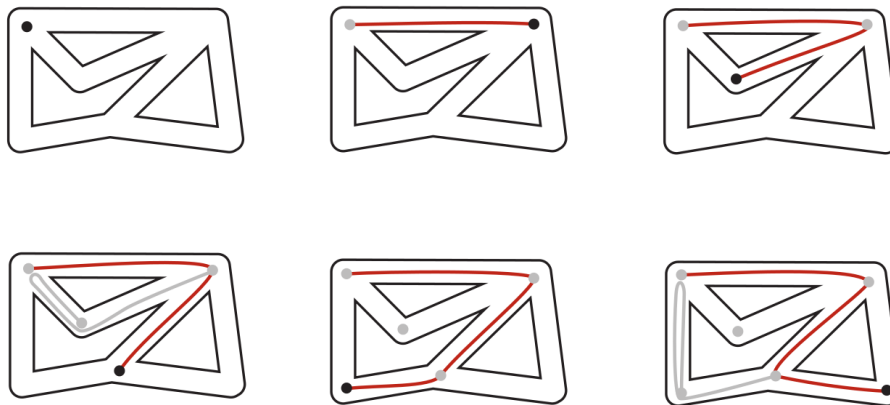
$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix} \times \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 3 & 1 & 0 & 1 \\ 1 & 2 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 2 \end{pmatrix}$$

Teorema. Siano poi v, w due generici vertici (anche coincidenti fra loro) del grafo rappresentato con la matrice M .

Per ogni numero naturale n , il numero di cammini di lunghezza n aventi per estremi i vertici v, w coincide con l'elemento della matrice potenza M^n all'incrocio fra la riga corrispondente al vertice v e la colonna corrispondente al vertice w .

Esplorazione di un grafo: Depth First Search

- + E' equivalente all'esplorazione di un labirinto, il cui obiettivo sia quello di visitare ogni intersezione (vertice).
- + Esplorazione di Trémaux:
 - + srotola un gomitolo mentre avanzi;
 - + marca ogni vertice e tieni traccia dell'arco che ad esso conduce;
 - + torna indietro quando non ci sono opzioni non visitate.



Depth First Search

- + Marca v come visitato;
- + Visita ricorsivamente tutti i vertici w non marcati adiacenti a v ;

```
public class DepthFirstSearch
{
    private boolean[] marked;
    private int count;

    public DepthFirstSearch(Graph G, int s)
    {
        marked = new boolean[G.V()];
        dfs(G, s);
    }

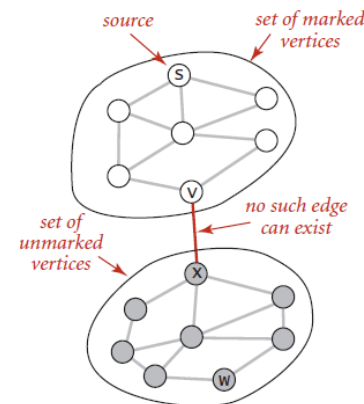
    private void dfs(Graph G, int v)
    {
        marked[v] = true;
        count++;
        for (int w : G.adj(v))
            if (!marked[w]) dfs(G, w);
    }

    public boolean marked(int w)
    { return marked[w]; }

    public int count()
    { return count; }
}
```

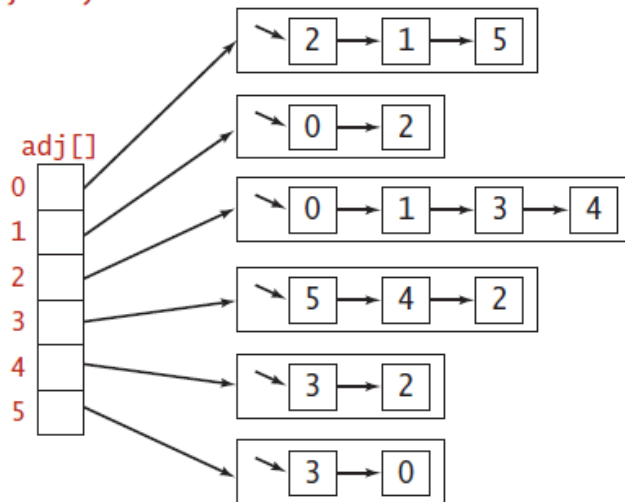
DFS: proprietà

- + **TEOREMA:** DFS esegue la marcatura di tutti i vertici connessi ad una sorgente s in tempo proporzionale alla somma dei loro gradi.
- + **DIMOSTRAZIONE:** L'algoritmo trova i vertici seguendo archi a partire da s e quindi, se w è marcato allora è connesso a s . Se w è connesso a s allora risulta marcato: se non fosse marcato, ogni cammino da s (che è marcato) a w avrebbe un arco da un nodo marcato v a un nodo non marcato x , ma l'algoritmo lo avrebbe trovato e marcato.
La marcatura assicura che ogni vertice connesso a s è visitato una volta, e il controllo di marcatura richiede tempo proporzionale al grado.
- + **OSSERVAZIONE:** Se tutti i vertici sono connessi a s , allora il tempo è proporzionale a $|E|$.

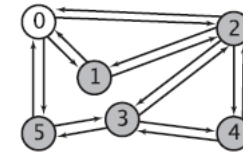


Esecuzione di DFS

adjacency lists



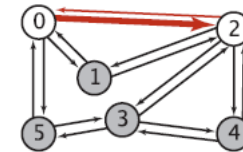
dfs(0)



marked[]	adj[]
0 T	0 2 1 5
1 T	1 0 2
2 T	2 0 1 3 4
3 T	3 5 4 2
4 T	4 3 2
5 T	5 3 0

dfs(2)

check 0



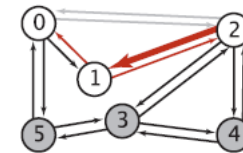
marked[]	adj[]
0 T	0 2 1 5
1 T	1 0 2
2 T	2 0 1 3 4
3 T	3 5 4 2
4 T	4 3 2
5 T	5 3 0

dfs(1)

check 0

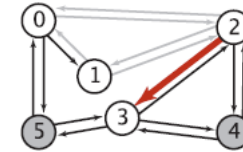
check 2

1 done



marked[]	adj[]
0 T	0 2 1 5
1 T	1 0 2
2 T	2 0 1 3 4
3 T	3 5 4 2
4 T	4 3 2
5 T	5 3 0

dfs(3)



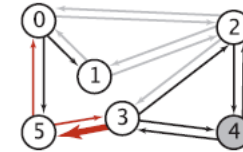
marked[]	adj[]
0 T	0 2 1 5
1 T	1 0 2
2 T	2 0 1 3 4
3 T	3 5 4 2
4 T	4 3 2
5 T	5 3 0

dfs(5)

check 3

check 0

5 done



marked[]	adj[]
0 T	0 2 1 5
1 T	1 0 2
2 T	2 0 1 3 4
3 T	3 5 4 2
4 T	4 3 2
5 T	5 3 0

dfs(4)

check 3

check 2

4 done

check 2

3 done

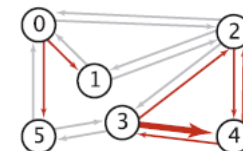
check 4

2 done

check 1

check 5

0 done



marked[]	adj[]
0 T	0 2 1 5
1 T	1 0 2
2 T	2 0 1 3 4
3 T	3 5 4 2
4 T	4 3 2
5 T	5 3 0

Trace of depth-first search to find vertices connected to 0

Problemi sui grafi

- + Possiamo affrontare due problemi:
 - + Connettività: Dati due dati vertici del grafo, essi sono connessi?
 - + Single source path: Dato un grafo e un nodo sorgente, esiste un path da s ad un dato vertice v ?

Generica classe per trovare cammini

```
public class Paths
```

```
    Paths(Graph G, int s) find paths in G from source s
```

```
    boolean hasPathTo(int v) is there a path from s to v?
```

```
    Iterable<Integer> pathTo(int v) path from s to v; null if no such path
```

```
public static void main(String[] args)
{
    Graph G = new Graph(new In(args[0]));
    int s = Integer.parseInt(args[1]);
    Paths search = new Paths(G, s);
    for (int v = 0; v < G.V(); v++)
    {
        StdOut.print(s + " to " + v + ": ");
        if (search.hasPathTo(v))
            for (int x : search.pathTo(v))
                if (x == s) StdOut.print(x);
                else StdOut.print("-" + x);
        StdOut.println();
    }
}
```

DepthFirstPaths: implementazione per trovare i cammini

```
public class DepthFirstPaths {
    private boolean[] marked; // s-v path?
    private int[] edgeTo;     // last edge on path
    private final int s;      // source vertex

    public DepthFirstPaths(Graph G, int s) {
        this.s = s;
        edgeTo = new int[G.V()];
        marked = new boolean[G.V()];
        dfs(G, s);
    }

    private void dfs(Graph G, int v) {
        marked[v] = true;
        for (int w : G.adj(v)) {
            if (!marked[w]) {
                edgeTo[w] = v;
                dfs(G, w);
            }
        }
    }

    public boolean hasPathTo(int v) {
        return marked[v];
    }

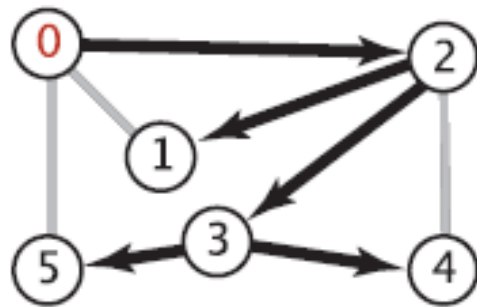
    public Iterable<Integer> pathTo(int v) {
        if (!hasPathTo(v)) return null;
        Stack<Integer> path = new Stack<Integer>();
        for (int x = v; x != s; x = edgeTo[x])
            path.push(x);
        path.push(s);
        return path;
    }

    public static void main(String[] args) {
        In in = new In(args[0]);
        Graph G = new Graph(in);
        int s = Integer.parseInt(args[1]);
        DepthFirstPaths dfs =
            new DepthFirstPaths(G, s);

        for (int v = 0; v < G.V(); v++) {
            if (dfs.hasPathTo(v)) {
                StdOut.printf("%d to %d: ", s, v);
                for (int x : dfs.pathTo(v)) {
                    if (x == s) StdOut.print(x);
                    else StdOut.print(" " + x);
                }
                StdOut.println();
            }
            else {
                StdOut.printf("%d to %d: not connected\n",
                               s, v);
            }
        }
    }
}
```

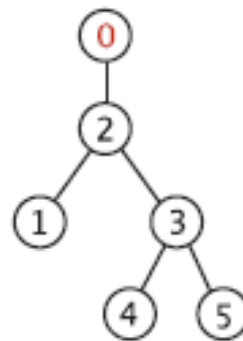
edgeTo[] è un array tale che edgeTo[w] = v
significa che v-w è l'arco usato per accedere per
la prima volta a w

Esempio



edgeTo[]

0	
1	2
2	0
3	2
4	3
5	3



x	path
5	5
3	3 5
2	2 3 5
0	0 2 3 5

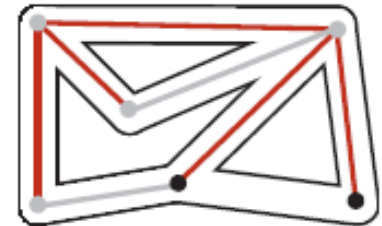
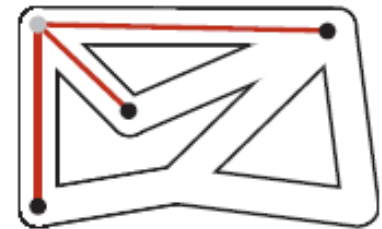
Trace of pathTo(5) computation

DFS: proprietà

- + **TEOREMA:** Una volta eseguita la DFS, si può trovare un percorso (se esiste) da s a v in tempo proporzionale alla sua lunghezza
- + **DIMOSTRAZIONE:** `edgeTo[]` è una rappresentazione nodo-padre di un albero con radice in s . `pathTo` costruisce il percorso da s a v in tempo proporzionale alla sua lunghezza.

Breadth first search

- + E' equivalente all'esplorazione di un labirinto effettuata in gruppo, con divisioni e ricongiungimenti in presenza di incroci.
- + BFS è adatto alla ricerca del cammino più breve da un vertice alla sorgente (shortest path).
- + Se DFS usa implicitamente uno stack per realizzare la ricorsione, BFS usa esplicitamente una coda per mantenere un ordine di distanza dei vertici dalla sorgente.



BFS

Estrai v dalla coda e visitalo;

Metti in coda e marca tutti i vertici w
non marcati adiacenti a v ;

Ripeti finchè la coda non si vuota

- + Arricchiamo la classe con i metodi necessari per trovare i cammini tra due nodi connessi
- + Il prodotto della BFS è un array `edgeTo[]` che fornisce una rappresentazione nodo-padre di un albero con radice nella sorgente s , il quale contiene i cammini minimi da s a ogni altro nodo connesso.

Breadth First Paths: implementazione

```
public class BreadthFirstPaths {
    private static final int INFINITY =
        Integer.MAX_VALUE;
    private boolean[] marked; // s-v path?
    private int[] edgeTo;
    private int[] distTo;

    public BreadthFirstPaths(Graph G, int s) {
        marked = new boolean[G.V()];
        distTo = new int[G.V()];
        edgeTo = new int[G.V()];
        bfs(G, s);
    }

    private void bfs(Graph G, int s) {
        Queue<Integer> q = new Queue<Integer>();
        for (int v = 0; v < G.V(); v++)
            distTo[v] = INFINITY;
        distTo[s] = 0;
        marked[s] = true;
        q.enqueue(s);
        while (!q.isEmpty()) {
            int v = q.dequeue();
            for (int w : G.adj(v)) {
                if (!marked[w]) {
                    edgeTo[w] = v;
                    distTo[w] = distTo[v] + 1;
                    marked[w] = true;
                    q.enqueue(w);
                }
            }
        }

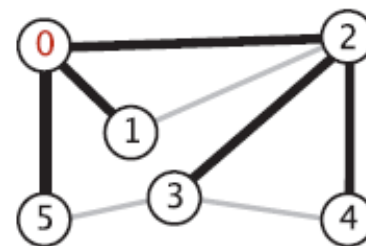
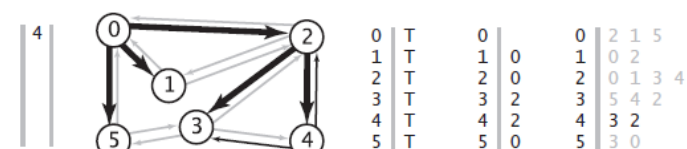
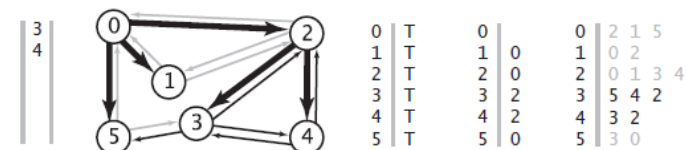
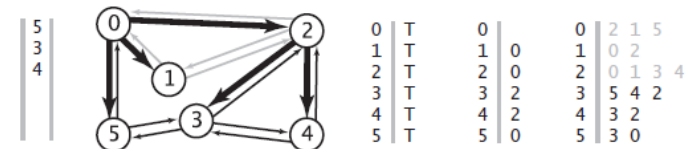
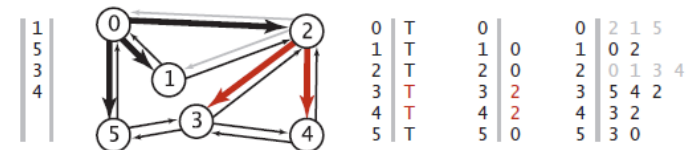
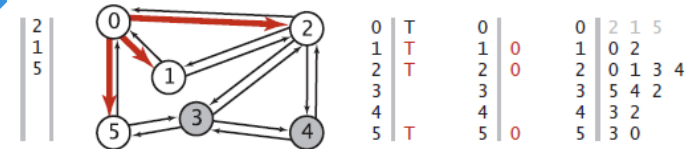
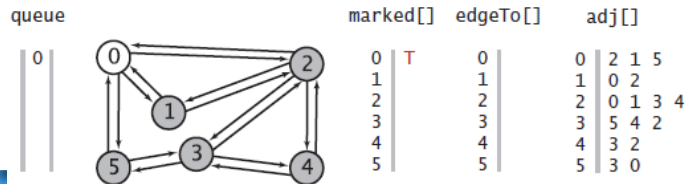
        public boolean hasPathTo(int v) {
            return marked[v];
        }

        public int distTo(int v) {
            return distTo[v];
        }

        public Iterable<Integer> pathTo(int v) {
            if (!hasPathTo(v)) return null;
            Stack<Integer> path = new Stack<Integer>();
            for (int x = v; distTo[x] != 0; x = edgeTo[x])
                path.push(x);
            path.push(s);
            return path;
        }

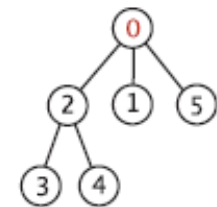
    public static void main(String[] args) {
        In in = new In(args[0]);
        Graph G = new Graph(in);
        int s = Integer.parseInt(args[1]);
        BreadthFirstPaths bfs = new
            BreadthFirstPaths(G, s);
        for (int v = 0; v < G.V(); v++) {
            if (bfs.hasPathTo(v)) {
                StdOut.printf("%d to %d (%d): ",
                    s, v, bfs.distTo(v));
                for (int x : bfs.pathTo(v)) {
                    if (x == s) StdOut.print(x);
                    else StdOut.print(" " + x);
                }
                StdOut.println();
            }
            else { ... }
        }
    }
}
```

Esempio



edgeTo[]

0	
1	0
2	0
3	2
4	2
5	0



Outcome of breadth-first search to find all paths from 0

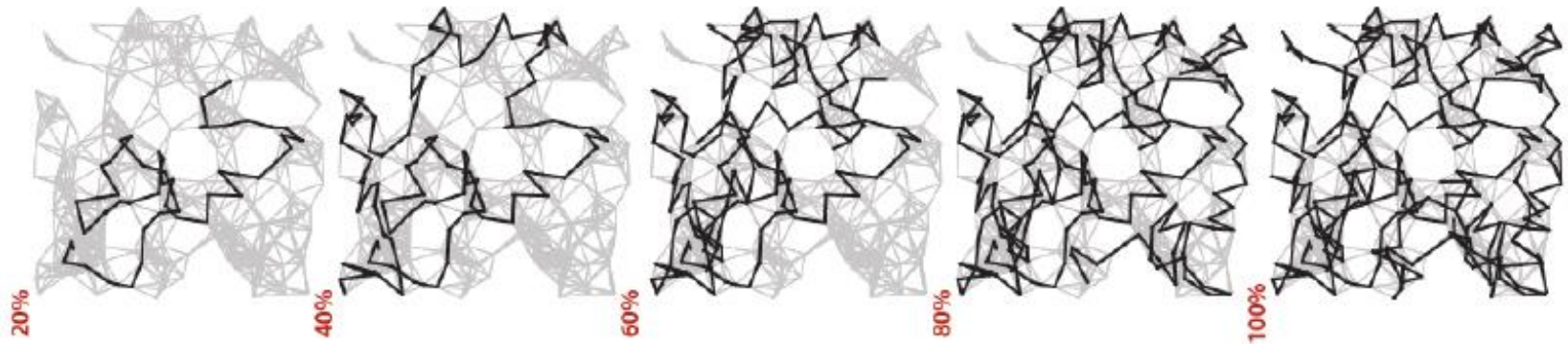
Trace of breadth-first search to find all paths from 0

BFS: proprietà

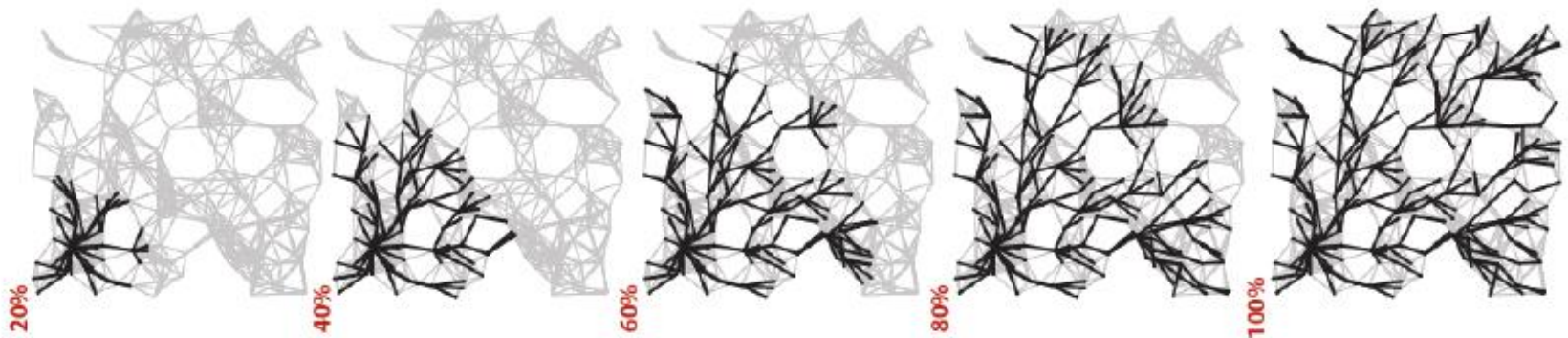
- + TEOREMA: BFS calcola i shortest path da s a tutti gli altri vertici del grafo in tempo proporzionale a $E + V$.
- + DIMOSTRAZIONE: Per un dato intero $k > 0$, la coda contiene sempre zero o più vertici di distanza k dalla sorgente, seguiti da zero o più vertici di distanza $k + 1$. I vertici entrano nella coda e lasciano la coda in ordine rispetto alla loro distanza dalla sorgente, e quindi l'algoritmo è corretto.
La marcatura assicura che ogni vertice connesso a s è visitato una volta, e il controllo di marcatura chiede tempo proporzionale al grado.

DFS vs BFS

stack



queue



Componenti connesse in un grafo

- + Scrivere una classe per trovare le componenti connesse di un grafo utilizzando la DFS

<pre>public class CC</pre>	
<pre> CC(Graph G)</pre>	<i>preprocessing constructor</i>
<pre> boolean connected(int v, int w)</pre>	<i>are v and w connected?</i>
<pre> int count()</pre>	<i>number of connected components</i>
<pre> int id(int v)</pre>	<i>component identifier for v (between 0 and count()-1)</i>

- + **Proposizione.** DFS in tempo e spazio proporzionale a $V + E$ consente di trovare le componenti connesse in un grafo non orientate.

Test client

```
public static void main(String[] args)
{
    Graph G = new Graph(new In(args[0]));
    CC cc = new CC(G);

    int M = cc.count();
    StdOut.println(M + " components");

    Bag<Integer>[] components;
    components = (Bag<Integer>[]) new Bag[M];
    for (int i = 0; i < M; i++)
        components[i] = new Bag<Integer>();
    for (int v = 0; v < G.V(); v++)
        components[cc.id(v)].add(v);
    for (int i = 0; i < M; i++)
    {
        for (int v: components[i])
            StdOut.print(v + " ");
        StdOut.println();
    }
}
```

Come opera la DFS?

DFS e componenti connesse

```
public class CC
{
    private boolean[] marked;
    private int[] id;
    private int count;

    public CC(Graph G)
    {
        marked = new boolean[G.V()];
        id = new int[G.V()];
        for (int s = 0; s < G.V(); s++)
            if (!marked[s])
            {
                dfs(G, s);
                count++;
            }
    }

    private void dfs(Graph G, int v)
    {
        marked[v] = true;
        id[v] = count;
        for (int w : G.adj(v))
            if (!marked[w])
                dfs(G, w);
    }
}
```

```
public boolean connected(int v, int w)
{ return id[v] == id[w]; }

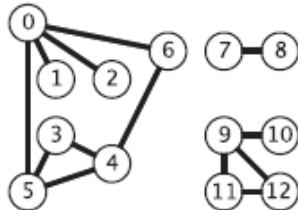
public int id(int v)
{ return id[v]; }

public int count()
{ return count; }
}
```

```
% more tinyG.txt
13 vertices, 13 edges
0: 6 2 1 5
1: 0
2: 0
3: 5 4
4: 5 6 3
5: 3 4 0
6: 0 4
7: 8
8: 7
9: 11 10 12
10: 9
11: 9 12
12: 11 9
```

```
% java CC tinyG.txt
3 components
6 5 4 3 2 1 0
8 7
12 11 10 9
```

tinyG.txt



	count	marked[]													id[]												
		0	1	2	3	4	5	6	7	8	9	10	11	12	0	1	2	3	4	5	6	7	8	9	10	11	12
dfs(0)	0	T													0												
dfs(6)	0	T						T							0					0							
check 0																											
dfs(4)	0	T			T		T								0				0		0						
dfs(5)	0	T			T	T	T								0				0	0	0						
dfs(3)	0	T			T	T	T	T							0				0	0	0	0					
check 5																											
check 4																											
3 done																											
check 4																											
check 0																											
5 done																											
check 6																											
check 3																											
4 done																											
6 done																											
dfs(2)	0	T	T	T	T	T	T	T							0	0	0	0	0	0	0						
check 0																											
2 done																											
dfs(1)	0	T	T	T	T	T	T	T							0	0	0	0	0	0	0						
check 0																											
1 done																											
check 5																											
0 done																											
dfs(7)	1	T	T	T	T	T	T	T	T						0	0	0	0	0	0	0	0	0	0	1		
dfs(8)	1	T	T	T	T	T	T	T	T	T					0	0	0	0	0	0	0	0	0	1	1		
check 7																											
8 done																											
7 done																											



Esplorazione in Grafì orientati

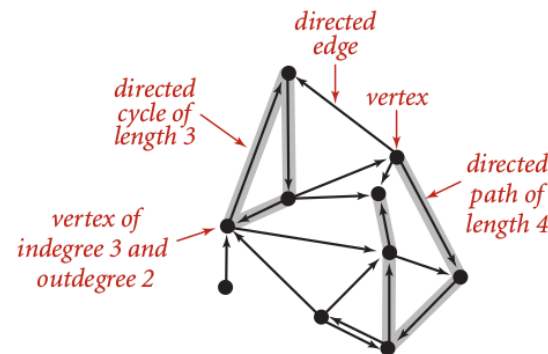
Grafo orientato

- + Un **Grafo** orientato (directed graph - digraph) è un insieme di vertici (o nodi) e una collezione di archi orientati (directed edges) che collegano **coppie ordinate** di nodi.

grafo	vertice	arco
web	pagina	hyperlink
catena alimentare	specie	predatore–preda
scheduling	task	precedenza
trasporto	incrocio	strada a senso unico
telefonia	telefono	chiamata
epidemiologia	persona	contagio
circuito	componente	input–output

Terminologia

- + Per ogni arco si distinguono **vertice di partenza** e **vertice di arrivo**.
- + **Grado entrante (indegree)** di un vertice v : numero di archi che hanno v come vertice di arrivo
- + **Grado uscente (outdegree)** di un vertice v : numero di archi che ha v come vertice di partenza
- + **Cammino orientato** (directed path): sequenza di vertici nella quale c'è un arco orientato da ogni vertice al suo successore nella sequenza; analogamente si può definire come una sequenza di archi tale che per ogni coppia di archi consecutivi u e v , il vertice di arrivo di u è anche vertice di partenza per v
- + **Ciclo orientato**: cammino orientato in cui il primo e l'ultimo vertice coincidono. Il ciclo è semplice se non ci sono archi o vertici ripetuti (esclusi il primo e l'ultimo). La sua lunghezza è il numero di archi.



Proprietà

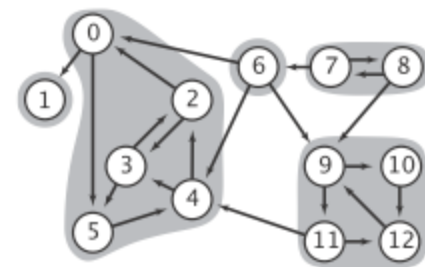
+ Dato un grafo orientato G con m archi e n vertici

1. $\sum_{v \in V} \text{indeg}(v) = \sum_{v \in V} \text{outdeg}(v) = ?$

Risposta: m

Terminologia

- + Un vertice w è raggiungibile da un vertice v se c'è un cammino orientato da v a w . Ciò non implica nulla sulla raggiungibilità di v da w .
- + Due vertici v e w sono fortemente raggiungibili se sono mutualmente raggiungibili, ovvero esiste un cammino orientato da v a w e un cammino orientato da w a v .
- + Un grafo orientato è fortemente connesso se ogni vertice è fortemente connesso a tutti gli altri.
- + Un grafo orientato che non è fortemente connesso consiste di un insieme di componenti fortemente connesse, che sono sottografi massimali fortemente connessi.
- + Un grafo orientato è aciclico (DAG) se non ha cicli orientati.



A digraph and its strong components

Quale struttura dati?

+ E' molto simile a quella usata per i grafi non orientati

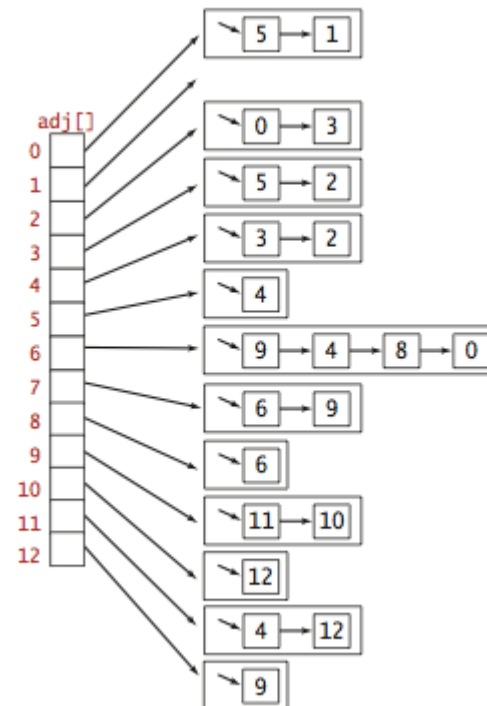
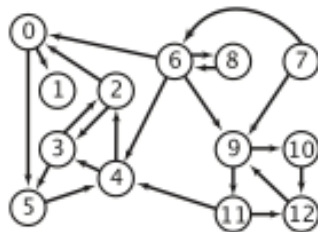
<code>public class Digraph</code>	
<code> Digraph(int V)</code>	<i>create a V-vertex digraph with no edges</i>
<code> Digraph(In in)</code>	<i>read a digraph from input stream in</i>
<code> int V()</code>	<i>number of vertices</i>
<code> int E()</code>	<i>number of edges</i>
<code> void addEdge(int v, int w)</code>	<i>add edge v->w to this digraph</i>
<code> Iterable<Integer> adj(int v)</code>	<i>vertices connected to v by edges pointing from v</i>
<code> Digraph reverse()</code>	<i>reverse of this digraph</i>
<code> String toString()</code>	<i>string representation</i>

Quale implementazione?

- + Così come per i grafi non orientati, si usa l'array delle adiacenze

tinyDG.txt

V → 13
22 ← E
4 2
2 3
3 2
6 0
0 1
2 0
11 12
12 9
9 10
9 11
7 9
10 12
11 4
4 3
3 5
6 8
8 6
5 4
0 5
6 4
6 9
7 6



Digraph client

```
In i n = new In ( args [ o ] );  
Digraph G = new Digraph ( i n );  
for ( int v = 0; v < G.V ( ) ; v++ )  
    for ( int w : G. adj ( v ) )  
        StdOut . println ( v + "-->" + w );
```


Spazio e tempo

- + Anche per il grafo orientato possono essere usate altre rappresentazioni

struttura dati	spazio	aggiungi arco	è w adiacente a v ?	itera sugli adiacenti a v
lista di archi	E	1	E	E
matrice di ad.	V^2	1	1	V
liste di ad.	$V + E$	1	$\text{outdeg}(v)$	$\text{outdeg}(v)$

Implementazione con array delle adiacenze

```
public class Digraph {  
    private final int V;  
    private Bag<Integer>[] adj;  
  
    public Digraph(int V) {  
        this.V = V;  
        adj = (Bag<Integer>[]) new Bag[V];  
        for (int v = 0; v < V; v++) {  
            adj[v] = new Bag<Integer>();  
        }  
    }  
  
    public void addEdge(int v, int w) {  
        adj[v].add(w);  
    }  
  
    public Iterable<Integer> adj(int v) {  
        return adj[v];  
    }  
}
```

Problemi di raggiungibilità nei grafi orientati

- + Raggiungibilità «a sorgente singola»: Dato un grafo orientato e un vertice sorgente s , esiste un cammino da s a v , dove v è un qualsiasi vertice del grafo?
 - + Si risolve usando l'esplorazione DFS sul grafo orientato
- + Raggiungibilità «a sorgente multipla»: Dato un grafo orientato e un insieme di vertici sorgente, esiste un cammino da ogni vertice dell'insieme a v , dove v è un qualsiasi vertice del grafo?
 - + Si risolve usando l'esplorazione DFS sul grafo orientato
- + Cammino orientato «a sorgente singola»: Dato un grafo orientato e un vertice sorgente s , esibire un cammino, se esiste, da s a v , dove v è un qualsiasi vertice del grafo?
 - + Si risolve usando l'esplorazione DFS sul grafo orientato
- + Cammino orientato più corto «a sorgente singola»: Dato un grafo orientato e un vertice sorgente s , esibire un cammino più corto, se esiste, da s a v , dove v è un qualsiasi vertice del grafo?
 - + Si risolve usando l'esplorazione BFS sul grafo orientato

Depth First Search sui grafi orientati

+ E' sostanzialmente uguale a DFS su grafi non orientati (o con tutti gli archi bi-orientati):

```
public class DirectedDFS
DirectedDFS(Digraph G, int s) // vertici
                                // raggiungibili da s
DirectedDFS(Digraph G, Iterable<Integer> sources)
                                // vertici raggiungibili
                                // da sources
boolean marked(int v)          // v e' raggiungibile?
```

Client:

```
public static void main (String[] args) {
    In in = new In(args[0]);
    Digraph G = new Digraph(in);
    Bag<Integer> sources = new Bag<Integer>();
    for (int i = 1; i < args.length; i++) {
        int s = Integer.parseInt(args[i]);
        sources.add(s);
    }

    DirectedDFS dfs = new DirectedDFS(G, sources);

    for (int v = 0; v < G.V(); v++) {
        if (dfs.marked(v)) StdOut.print(v + "_");
    }
    StdOut.println();
}
```

Depth first search sui grafi orientati

```
public class DirectedDFS {  
    private boolean[] marked; // true if reachable from source or sources  
    private int count;        // n. of vertices reachable from s  
  
    public DirectedDFS(Digraph G, int s) {  
        marked = new boolean[G.V()];  
        dfs(G, s);  
    }  
  
    public DirectedDFS(Digraph G, Iterable<Integer> sources) {  
        marked = new boolean[G.V()];  
        for (int v : sources) {  
            if (!marked[v]) dfs(G, v);  
        }  
    }  
  
    private void dfs(Digraph G, int v) {  
        count++;  
        marked[v] = true;  
        for (int w : G.adj(v)) {  
            if (!marked[w]) dfs(G, w);  
        }  
    }  
  
    public boolean marked(int v) {  
        return marked[v];  
    }  
  
    public int count() {  
        return count;  
    }  
}
```

DFS: proprietà

- + **TEOREMA:** DFS esegue la marcatura di tutti i vertici raggiungibili da un dato insieme S di sorgenti in un tempo proporzionale alla somma degli outdegree dei vertici marcati.
- + **DIMOSTRAZIONE:** L'algoritmo trova i vertici seguendo archi a partire da S e quindi, se w è marcato allora è raggiungibile da un s che appartiene a S . Se w è raggiungibile da s appartenente a S allora risulta marcato: se non fosse marcato, ogni cammino da s (che è marcato) a w avrebbe un arco da un nodo marcato v a un nodo non marcato x , ma l'algoritmo lo avrebbe trovato e marcato. La marcatura assicura che ogni vertice raggiungibile da S è visitato una volta, e il controllo di marcatura chiede tempo proporzionale al grado.

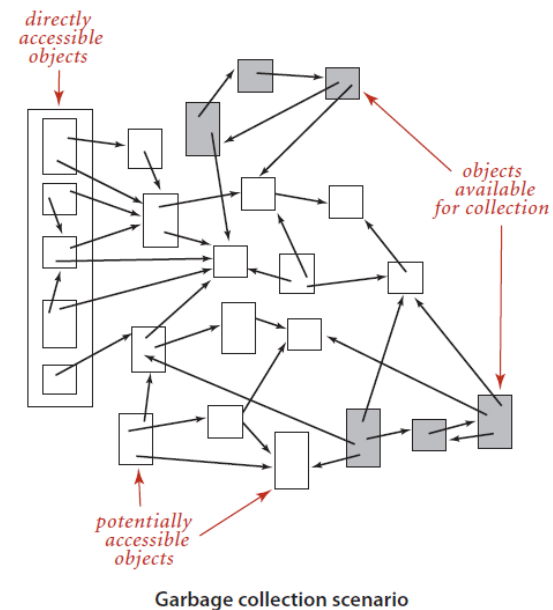
Raggiungibilità: applicazioni

- + Le applicazioni sono numerose: tra queste citiamo il controllo del flusso dei programmi.
- + Ogni programma è un grafo orientato:
 - + Nodo: blocco di istruzioni
 - + Arco: salto da un blocco all'altro
- + Eliminazione di codice morto: blocchi non raggiungibili.
- + Individuazione di loop infiniti: irraggiungibilità del blocco di uscita.

Raggiungibilità: applicazioni

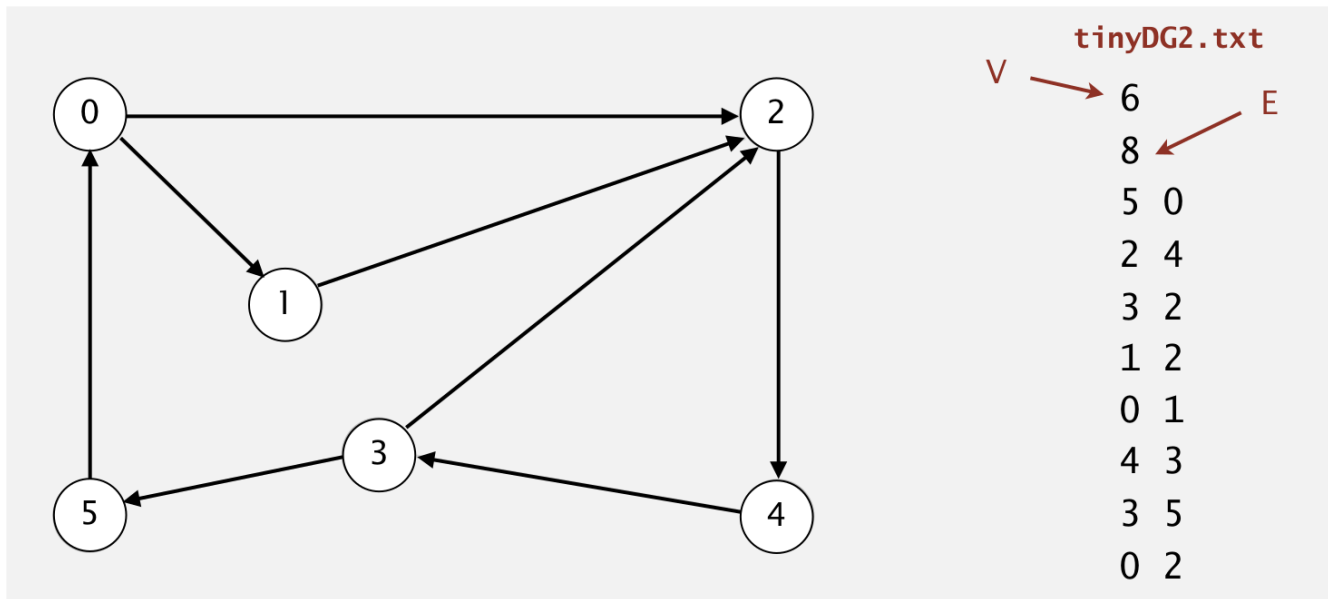
Garbage collection

- + Ogni struttura dati è un grafo orientato:
 - + Nodo: oggetto
 - + Arco: riferimento a oggetto
- + Oggetti raggiungibili: sono quelli accessibili indirettamente dal programma attraverso catene di riferimenti.
- + Algoritmo mark-sweep [McCarthy, 1960]
 - + Mark: marca tutti gli oggetti raggiungibili;
 - + Sweep: se un oggetto non è marcato, mettilo a disposizione per essere liberato.



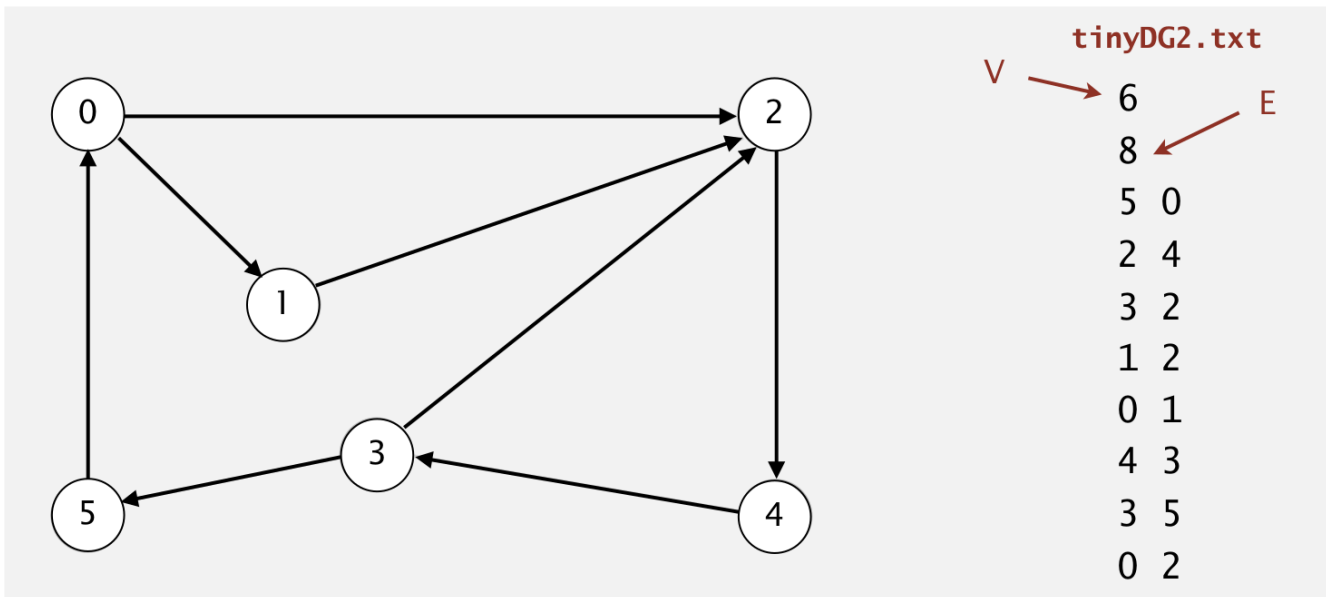
Esecuzione di DFS

+ Eseguire la visita DFS sul grafo orientato



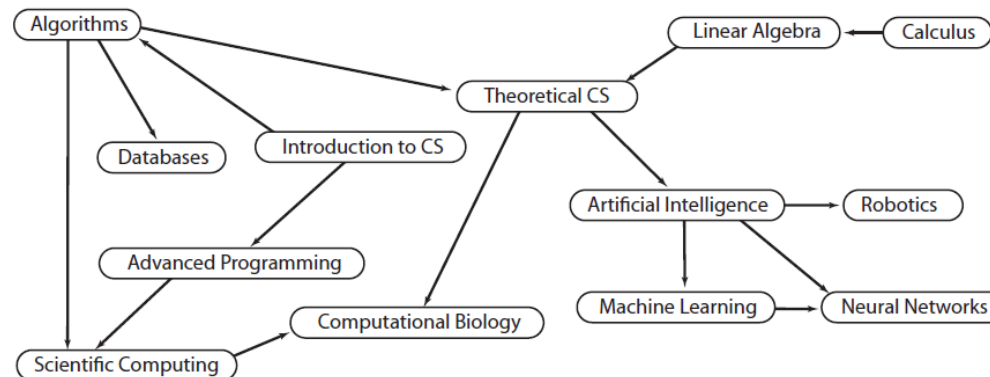
Breadth first search su grafi orientati

- + E' sostanzialmente uguale a BFS su grafi non orientati (o con tutti gli archi bi-orientati):
- + Eseguire la visita BFS sul grafo



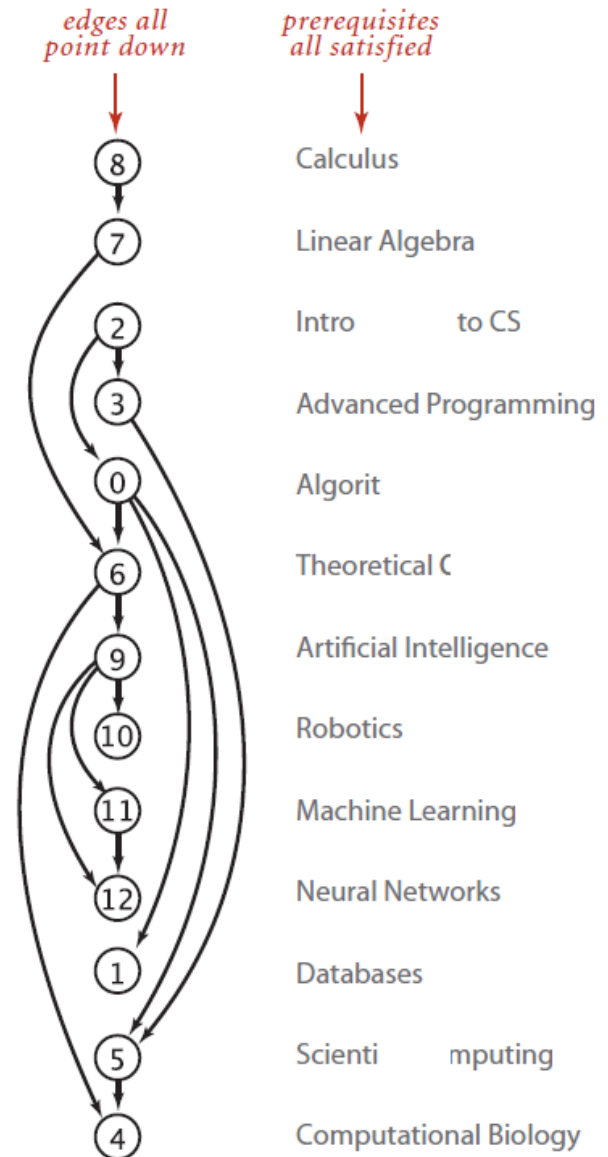
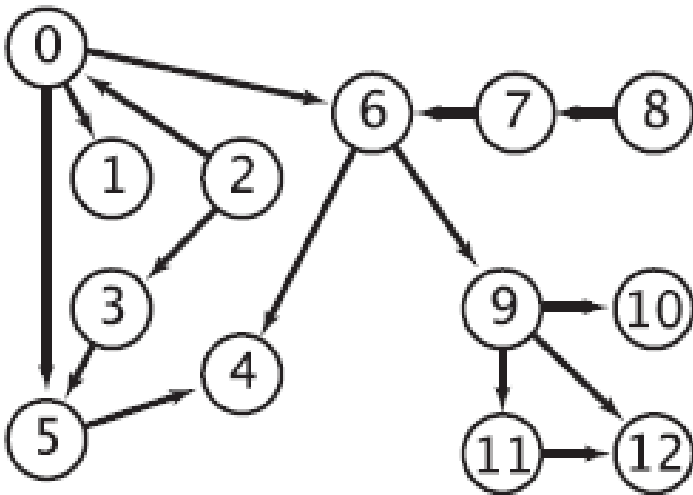
Applicazioni dei grafi: scheduling

- + **Obiettivo:** Dato un insieme di compiti (task) sottoposti a vincoli di precedenza, trovare l'ordine con cui tali compiti devono essere programmati (scheduled);
- + **Modello:** Grafo orientato in cui vertici sono associati ai task e gli archi rappresentano i vincoli di precedenza;
- + **Esempio:** vincoli di propedeuticità tra discipline



Ordinamento topologico di un grafo orientato

- + E' una sequenza dei vertici del grafo tale che tutti gli archi del grafo hanno un vertice di partenza che precede nella sequenza il vertice d'arrivo.



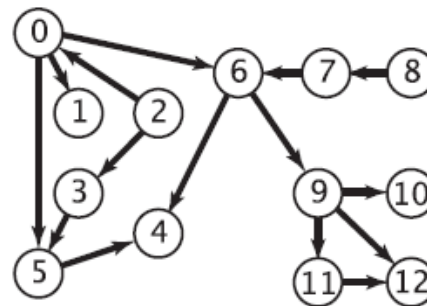
Topological sort

Ordinamento topologico e grafi aciclici

- + **Teorema:** Un grafo orientato ha un ordine topologico se e solo se è un DAG.
- + **Dimostrazione:** Se il grafo ha un ciclo orientato allora non ha un ordine topologico. Viceversa se il grafo è aciclico esiste un algoritmo che trova il suo ordine topologico.

Un algoritmo per l'ordinamento topologico

- + In generale possiamo definire tre ordini diversi di visita DFS di un grafo orientato
 - + *Preorder* : inserisci i vertici in una coda prima della chiamata ricorsiva
 - + *Postorder* : inserisci i vertici in una coda dopo la chiamata ricorsiva
 - + *Reverse postorder* : Inserisci i vertici in una pila dopo la chiamata ricorsiva.
- + DepthFirstOrder.java trova gli ordini precentemente descritti
- + Applichamole al grafo



preorder
is order of
dfs() calls

postorder
is order
in which
vertices
are done

	pre	post	reversePost
dfs(0)	0		
dfs(5)	0 5		
dfs(4)	0 5 4		
4 done		4	4
5 done		4 5	5 4
dfs(1)	0 5 4 1		
1 done		4 5 1	1 5 4
dfs(6)	0 5 4 1 6		
dfs(9)	0 5 4 1 6 9		
dfs(11)	0 5 4 1 6 9 11		
dfs(12)	0 5 4 1 6 9 11 12		
12 done		4 5 1 12	12 1 5 4
11 done		4 5 1 12 11	11 12 1 5 4
dfs(10)	0 5 4 1 6 9 11 12 10		
10 done		4 5 1 12 11 10	10 11 12 1 5 4
check 12		4 5 1 12 11 10 9	9 10 11 12 1 5 4
9 done		4 5 1 12 11 10 9 6	6 9 10 11 12 1 5 4
check 4		4 5 1 12 11 10 9 6 0	0 6 9 10 11 12 1 5 4
6 done			
0 done			
check 1			
dfs(2)	0 5 4 1 6 9 11 12 10 2		
check 0			
dfs(3)	0 5 4 1 6 9 11 12 10 2 3		
check 5		4 5 1 12 11 10 9 6 0 3	3 0 6 9 10 11 12 1 5 4
3 done		4 5 1 12 11 10 9 6 0 3 2	2 3 0 6 9 10 11 12 1 5 4
2 done			
check 3			
check 4			
check 5			
check 6			
dfs(7)	0 5 4 1 6 9 11 12 10 2 3 7		
check 6		4 5 1 12 11 10 9 6 0 3 2 7	7 2 3 0 6 9 10 11 12 1 5 4
7 done			
dfs(8)	0 5 4 1 6 9 11 12 10 2 3 7 8		
check 7		4 5 1 12 11 10 9 6 0 3 2 7 8	8 7 2 3 0 6 9 10 11 12 1 5 4
8 done			
check 9			
check 10			
check 11			
check 12			

reverse
postorder

Ordinamento topologico

- + Teorema: Il Reverse postorder in un DAG è un ordinamento topologico
- + Dimostrazione: Consideriamo un arco $v \rightarrow w$. Quando $\text{dfs}(v)$ è chiamata, uno dei seguenti tre casi si verifica:
 - + $\text{dfs}(w)$ è stata già chiamata e la sua esecuzione è terminata;
 - + $\text{dfs}(w)$ non è ancora stata chiamata (w è unmarked), quindi $v \rightarrow w$ provocherà la chiamata $\text{dfs}(w)$ (e return), prima del return di $\text{dfs}(v)$;
 - + $\text{dfs}(w)$ è stata già chiamata ma non ha terminato la sua esecuzione. Quest'ultimo caso è impossibile poichè implicherebbe l'esistenza di un cammino da w a v , cosa che contraddirebbe il fatto di essere in un DAG. Negli altri due casi possibili, $\text{dfs}(w)$ si concluderà prima di $\text{dfs}(v)$, pertanto w appare prima di v nella vista in postordine e dopo v nel reverse postorder. Quindi l'arco $v \rightarrow w$ avrà un vertice di partenza che precede nell'ordine il vertice d'arrivo, come desiderato.

Testare se un grafo orientato è aciclico

- + Rilevamento di eventuali cicli:
 - + esegui DFS, tenendo traccia di quali nodi stanno nello stack della ricorsione;
 - + in presenza di un arco $v \rightarrow w$, verifica se w è nello stack.
- + L'ordinamento topologico può essere eseguito in tempo $O(V+E)$

Implementazione

```
public class DirectedCycle {
    private boolean[] marked;           // marked[v] = has vertex v been marked?
    private int[] edgeTo;               // edgeTo[v] = previous vertex on path to v
    private boolean[] onStack;         // onStack[v] = is vertex on the stack?
    private Stack<Integer> cycle;       // directed cycle (or null if no such cycle)

    public DirectedCycle(Digraph G) {
        marked = new boolean[G.V()];
        onStack = new boolean[G.V()];
        edgeTo = new int[G.V()];
        for (int v = 0; v < G.V(); v++)
            if (!marked[v] && cycle == null) dfs(G, v);
    }

    private void dfs(Digraph G, int v) {
        onStack[v] = true;
        marked[v] = true;
        for (int w : G.adj(v)) {
            if (cycle != null) return;
            else if (!marked[w]) {
                edgeTo[w] = v; dfs(G, w); }
            else if (onStack[w]) {
                cycle = new Stack<Integer>();
                for (int x = v; x != w; x = edgeTo[x])
                    cycle.push(x);
                cycle.push(w);
                cycle.push(v);
            }
        }
        onStack[v] = false;
    }
    ...
}
```